

NSI Terminale Spé | Système de base de données pour l'utilisation de QCM en ligne

PLE Jules

IRDEL Jb

Mai 2026

1 Problématique :

Comment mettre en place un système de base de données opérable avec un site web pour permettre aux utilisateurs d'interagir avec des QCMs ?

2 Introduction :

Le programme que nous présentons relie une base de données SQL à un site web pour permettre une rétention des données sur un serveur à travers le temps. Tandis que le client navigue sur le site, ses données sont stockées sur le serveur avec SQL.

3 Aspect théorique :

On distingue le backend et le frontend dans la réalisation d'un site web : Le backend consiste en la gestion des données. Ici le backend est opéré par le système SQL pour les bases de données. Le frontend consiste à faire interagir l'utilisateur avec le site. Dans notre cas le site web présente deux accès :

- un accès étudiant : avec lequel il a la possibilité d'accéder aux QCMs disponibles
- et un accès professeur permettant de supprimer et d'ajouter des QCMs

4 Part sociétale :

Pourquoi avoir réalisé ce projet ?

Il est utile pour un établissement scolaire de se reposer sur une alternative open source plutôt qu'un site/logiciel fermé. En effet bien qu'il existe des alternatives comme Quizziz, pour les établissements scolaires la gestion des données requiert d'une importance considérable. Il peut donc être idéal dans certains cas de faire tourner le serveur sur le réseau de l'établissement afin de garder une forme d'indépendance sur la gestion des données de chaque étudiants et de chaque professeurs.

5 Partie algorithmique

5.1 Pour le backend / le site web

- Fonction `ouvrir_connexion_bd` : permet d'ouvrir une connexion à la base de données MySQL en `utf8mb4`.

- Fonction `menu` : génère le menu de navigation du site selon le rôle de l'utilisateur (visiteur, étudiant ou professeur).
- Fonction `index` : instancie la page d'accueil du site et affiche un message différent selon que l'utilisateur est connecté ou non.
- Fonction `moteurqcm` : gère le déroulement du QCM. Elle récupère la matière et la classe choisies, affiche les questions, vérifie les réponses de l'utilisateur, calcule la note et enregistre le score dans la base de données lorsque le QCM est terminé.
- Fonction `seconde` : affiche les matières disponibles pour les élèves de seconde et initialise les variables de session du QCM.
- Fonction `premiere` : affiche les matières disponibles pour les élèves de première et initialise les variables de session du QCM.
- Fonction `terminale` : affiche les matières disponibles pour les élèves de terminale et initialise les variables de session du QCM.
- Fonction `ajout_qcm` : permet à un professeur d'ajouter une nouvelle question de QCM dans la base de données.
- Fonction `sup` : permet à un professeur de rechercher une question puis de la supprimer de la base de données.
- Fonction `connection` : permet à un utilisateur de se connecter. Elle vérifie l'identifiant, compare le mot de passe avec celui stocké dans la base de données et ouvre une session.
- Fonction `deconnexion` : permet de déconnecter l'utilisateur en supprimant les informations de session puis en le redirigeant vers l'accueil.
- Fonction `creer_compte` : permet de créer un nouveau compte utilisateur. Le mot de passe est haché avec `bcrypt` avant d'être enregistré dans la base de données.
- Fonction `stats` : récupère et affiche les statistiques de l'utilisateur connecté, c'est-à-dire ses notes selon les matières et les classes.

5.2 Pour la base de données

- Table `utilisateur` : stocke les identifiants, mots de passe et rôles des utilisateurs.
- Table `questions` : stocke les questions, les quatre réponses possibles, la bonne réponse, la matière et la classe concernée.
- Table `notes` : stocke les résultats obtenus par chaque utilisateur pour une matière et une classe données.

quel type de donnée à utiliser pour représenter la classe dans la base SQL ? Nous avons réalisé deux standards pour la base de données du QCM :

- l'un utilisait les chiffres 2, 1, 0 pour représenter les classes seconde, première, terminale
- l'autre utilisait les chaînes de caractères 'seconde', 'première', 'terminale' pour représenter également les classes. Le premier standard est resté puisqu'il offre une légère économie de stockage ainsi qu'une meilleure stabilité.

comment ordonner les différentes pages ? Au départ, notre moteur pour afficher le qcm se répartissait sur les fonctions `seconde`, `premiere` et `terminale`.

Néanmoins on observait rapidement qu'il existait une redondance des lignes de codes sur ces fonctions. Cette redondance augmentait significativement la complexité d'analyse de notre code ainsi que la gestion des erreurs.

En conséquence, une fonction `moteurqcm` a été créée, elle avait pour objet de s'occuper de la gestion de l'interface du qcm. Dès lors, nous avons identifié de nombreuses erreurs qui ont été rapidement corrigés puisque notre code ne se trouvait plus dans 3 fonctions mais dans une seule.

En terminale si nous avons la possibilité d'optimiser notre algorithme alors nous pouvons le faire.

6 Ressources documentaire :

- GitHub (documentation et exemples de projets similaires)
- Documentation Cherrypy pour utiliser cherrypy
- <https://sql.sh/cours/select> : documentation SQL
- <https://developer.mozilla.org/fr/docs/Web/HTML/Reference/Elements/button> : pour les boutons html
- https://developer.mozilla.org/fr/docs/Learn_web_development/Extensions/Forms/Your_first_form pour les spécificités concernant les formulaires html
- <https://github.com/Sc00bz/bscrypt> : github de l'algorithme de cryptage utilisé

7 Part projective et part minimale :

- Part minimale :
 - développement d'une base de données contenant les QCMs et les différents comptes
 - développement d'un site web permettant d'accéder aux QCMs
- Part projective :
 - permettre d'ajouter des images associées au QCM
 - étendre la connexion réseau pour internet

8 Outils logiciel :

- Cherrypy : bibliothèque permettant la gestion de site web
- os : nécessaire au fonctionnement de cherrypy
- bcrypt : pour le cryptage des mots de passe
- mysql.connector : permet la connexion entre la base de donnée SQL et le site

9 Matériels nécessaires :

- 1 ordinateur (serveur)

10 Méthodologie :

1. Le choix de la base de données Il a été choisi tout d'abord de créer une première base dans un fichier unique **setup.sql** dans le but de l'étendre à mesure que nous rencontrons des difficultés. Cela n'aurait pas été possible si nous avions utilisé des commandes dans un terminal, En effet, nous aurions dû noter chaque commande et les exécuter lignes par lignes, rendant notre base de code significativement plus complexe à gérer.

Nous avons créé 3 tables dès le premier jour, *l'architecture peut être observée sur le schéma*. Et nous avons utilisé la commande **SOURCE** afin de mettre à jour la base sur chaque changement du fichier setup.sql.

2. La création d'un prototype Nous avons choisi de créer un prototype avec python dans le fichier questionnaire_sql.py. Cette option nous permettait d'itérer plus rapidement les changements en commençant le projet sans site web. En réduisant le nombre de lignes à quelques requêtes SQL on pouvait facilement corriger les requêtes SQL erronées.

3. La mise à l'échelle sur le site Nous avons ensuite intégré le module python à un site web Cherrypy. Nous avons utilisé mysql.connector pour la connexion entre la base SQL et le site. Nous avons également intégré des fonctions de cryptage comme SHA1 que nous avons vu dans l'année afin de sécuriser les comptes des utilisateurs.

11 Repartition des tâches au sein du groupe :

Afin de permettre une meilleure maîtrise du projet nous ne voulions pas séparer le backend et le frontend. C'est ainsi que nous avons trouvé plus préférable de travailler tout les deux sur les parties backend et frontend.

12 Journal de Bord :

12.1 Jb

Semaine 1 ~ 9 - 15 mars :

~ *I Création de setup.sql afin d'instancier les tables sql pour le setup des fichiers sql, nous avons besoin de différents prérequis :*

1. Installation de Mysql ou Mariadb
2. Création d'une database PQCM02, avec

```
1 CREATE DATABASE PQCM02;
```

3. **instanciation du fichier setup.sql** le fichier setup.sql permet d'instancier toutes les tables de la database

```
1 USE PQCM02;  
2 SOURCE ./setup.sql;
```

une fois ces étapes réalisés dans le terminal avec mysql, nous pouvons utiliser nos QCMs depuis la base de données

~ *II Création de questionnaire_sql.py afin de tester notre database avec un petit programme*

Semaine ~ 6 - 12 avril :

~ *III Application de questionnaire_sql.py au Site pour faire un QCM*

23 avril :

~ *IV Correction des bugs de connexion et fusion des codes*

14 mai amélioration de l'interface web :

~ *V Correction des bugs de connexion et fusion des codes*

12.2 Jules

12.2.1 Journée du 7-8 mars

Creation du site : Creation du bouton pour sélectionner la classe voulu. Creation du bouton pour se connecter et créer un compte.

12.2.2 Journée du 13 mars :

Installation d'enregistrement en sql dans creer_compte

12.2.3 Journée du 29 mars :

Ajout de la partie de cryptage puis recherche du bon mdp dans la base de donnée dans la fonction connection

12.2.4 Semaine du 12 avril :

Modification de la partie cryptage avec la mise en place de la bibliotheque bcrypt qui hach le mdp Puis mise en place d'un debut de chopse pour l'autentification directement sur le site

12.2.5 Samedi 18 avril :

Ajout de deux fonctions une qui permet d'ajouter des qcms si l'on est connecté et en tant que professeur puis une autre qui permet de supprimer des qcms si l'on est connecter en tant que professeur aussi. J'ai pu aussi mettre une condition dans le creer_compte avec que si le nom de user donné par la personne qui veut créer un compte est deja prise alors l'utilisateur qui crée le compte est signalé que le nom est deja prit. J'ai aussi donc mis en place le system de session qui permet de restreindre certaines parties du sites.

12.2.6 18 mai :

Correction de plusieurs choses dans le programme python liée au qcm, pour les classes 2nd, 1ere et Terminale on peut aller chercher les questions du niveau. Puis modification du css pour faire que quand l'on passe la souris dessus l'onglet classe les classes apparaissent en dessous.

Schéma relationnel de la base de données

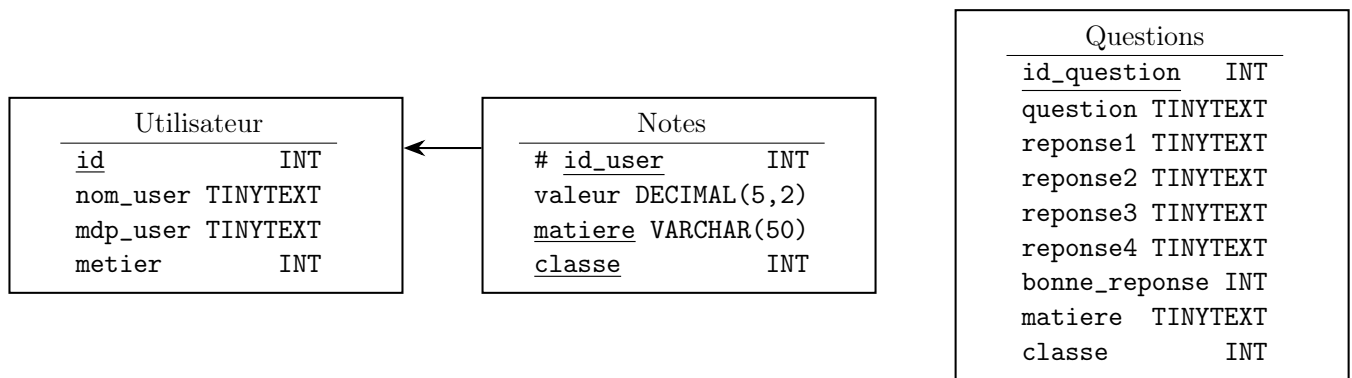


Schéma relationnel

Utilisateur(id, nom_user, mdp_user, metier)

Notes(#id_user, valeur, matiere, classe)

Questions(id_question, question, reponse1, reponse2, reponse3,
reponse4, bonne_reponse, matiere, classe)

Légende

<u>attribut</u>	clé primaire
# attribut	clé étrangère
# <u>attribut</u>	clé primaire et clé étrangère

-Nous avons souligné les clés primaires qui sont id pour utilisateur et id_user, matiere et classe la table notes.

On observe que la table notes opère un rôle central dans la connexion entre les utilisateur et les questions permettant d'établir de potentielles jointures (si nécessaire).

Au départ nous avons choisi une autre architecture avec la notation de chaque questions. Néanmoins elle a été abandonnée au profit d'une architecture moins gourmande en mémoire en ne notant que le pourcentage du QCM. Nous avons donc optimiser les ressources du serveur pour pouvoir servir à grande échelle si possible. Une notation pour chaque questions aurait consommé 10 fois plus de mémoire.

L'ancienne architecture nous permettait cependant une bien meilleure interconnection des tables et une meilleure précision puisque nous avons accès aux erreurs précises des utilisateurs.

Ancien Schéma relationnel

